

NEXATECH.

DOCUMENTACIÓN TÉCNICA

Proyecto web corporativo desarrollado con WordPress y PHP



Vantix Retail



Stratum Solutions



Axionis Pharma

Proyecto de portfolio · Gaston Rizzo

Índice

N.º	Sección	Pág.
1	Descripción general	3
2	Funcionamiento general del proyecto	3
3	Tecnologías y dependencias	4
4	Base de datos y modelo de información	5
5	Estructura del proyecto	8
6	WordPress y el tema personalizado	8
7	Página de Inicio	10
8	Página de Servicios	11
9	Página de Proceso	11
10	Casos de estudio y artículos	12
11	Formulario de contacto	15
12	JavaScript e interacciones dinámicas	16
13	CSS y adaptación de la interfaz	16
14	Recursos multimedia y servicios externos	17
15	Caché y rendimiento	18
16	Tratamiento de datos y seguridad	18
17	Instalación y ejecución del proyecto	19
18	Posibles líneas de evolución	20
19	Alcance técnico y conclusión	20

1. Descripción general

NexaTech es un sitio web corporativo construido sobre WordPress que simula una consultora tecnológica B2B. El proyecto presenta servicios digitales, explica una metodología de trabajo, reúne casos de estudio y ofrece un formulario de contacto para potenciales clientes.

El frontend se construye mediante un tema personalizado llamado nexatech-theme, con templates PHP propios, una hoja de estilos central y JavaScript ejecutado en el navegador. WordPress aporta el ciclo de renderizado, el panel de administración, las páginas, la biblioteca de medios y la base de datos.

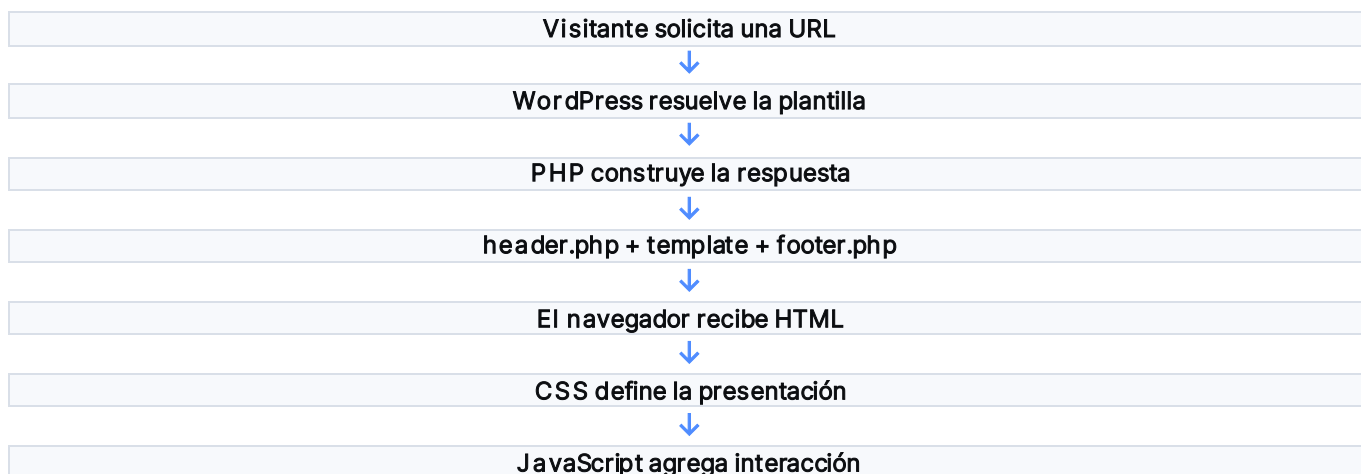
Área	Función dentro de NexaTech
Inicio	Presentación general, métricas, servicios, proceso y casos.
Servicios	Desarrollo de las áreas técnicas ofrecidas por la consultora.
Proceso	Timeline con cuatro fases y métricas animadas.
Casos de estudio	Carrusel de marcas, casos destacados y páginas de detalle administradas con WordPress.
Contacto	Formulario con validación HTML, sanitización PHP y estados de éxito/error.

Idea central

WordPress administra el entorno y parte del contenido; el tema personalizado controla la estructura HTML y la mayor parte de la presentación visible.

2. Funcionamiento general del proyecto

Cuando una persona solicita una dirección, WordPress determina qué plantilla debe resolverla. PHP se ejecuta en el servidor, arma el HTML y lo entrega al navegador. Después el navegador aplica nexatech.css y ejecuta el JavaScript propio y las librerías externas.



2.1 Ejemplos de resolución

Dirección / área	Template utilizado	Resultado
/	front-page.php	Inicio
/servicios/	page-servicios.php	Página Servicios
/proceso/	page-proceso.php	Página Proceso
/casos-de-estudio/	page-casos-de-estudio.php	Listado de casos
/contacto/	page-contacto.php	Formulario de contacto
Caso individual	page-articulo.php	Imagen destacada + contenido administrado desde WordPress

El encabezado y el pie no se copian manualmente en cada archivo. Las páginas llaman a `get_header()` y `get_footer()`, de modo que la navegación y el cierre global del sitio permanecen centralizados.

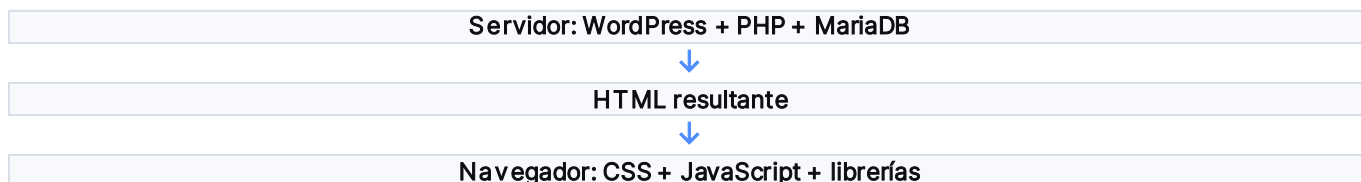
3. Tecnologías y dependencias

NexaTech utiliza recursos del lado del servidor, componentes que se ejecutan en el navegador y librerías externas. Los nombres técnicos se mantienen porque forman parte del código del proyecto, y cada uno cumple una función específica dentro del sitio.

Tecnología / dependencia	Uso real en NexaTech
WordPress	CMS, sistema de páginas, resolución de templates, biblioteca de medios y APIs del tema.
PHP	Construcción de templates, hooks del tema, formulario y redirecciones.
HTML5	Estructura de páginas, formularios, tarjetas, listas y navegación.
CSS3	Diseño, variables de color, grids, timeline, estados, animaciones y responsive.
JavaScript	Contadores, filtros, sliders y comportamiento dinámico.
MariaDB / MySQL	Persistencia utilizada por WordPress para páginas, opciones, usuarios y metadatos.
AOS.js 2.3.4	Animaciones de entrada al hacer scroll.
Typed.js 2.0.12	Texto escrito y borrado automáticamente en el hero de Inicio.
Isotope.js 3	Filtrado dinámico de casos en Inicio.
Splide.js 4.1.4 + Auto Scroll 0.5.3	Carrusel continuo de logos en Casos de estudio.
Google Fonts	Carga de la tipografía Inter desde un servicio externo.

3.1 Servidor y navegador

WordPress y PHP trabajan antes de que la página llegue al visitante. En cambio, CSS, JavaScript y las librerías frontend actúan en el navegador. Esta separación explica por qué el formulario se valida nuevamente en PHP aunque el HTML ya utilice required y type="email".



4. Base de datos y modelo de información

NexaTech utiliza la estructura estándar de WordPress. El tema no crea tablas propias ni un Custom Post Type específico para los casos. Las páginas, los attachments, los metadatos y las opciones se guardan mediante las tablas normales del CMS.

4.1 Datos generales de la instalación

Dato	Valor
Motor de base de datos	MariaDB 11.8.6
Prefijo de tablas	wp_
Tablas existentes	21
Tema activo	nexatech-theme / NexaTech Custom
Páginas publicadas	7
Attachments registrados	3
Custom Post Types propios	Ninguno
Plugins activos registrados	5: Hostinger AI Assistant, Easy Onboarding, Hostinger Reach, Hostinger y Media Sync

4.2 Tablas relevantes

Tabla	Función en el proyecto
wp_posts	Páginas, casos individuales, attachments y otros contenidos de WordPress.
wp_postmeta	Metadatos asociados a páginas y archivos; incluye template asignado e imagen destacada.
wp_options	URL del sitio, tema activo, plugins activos y configuración general.
wp_users / wp_usermeta	Cuenta administrativa y sus capacidades.
wp_terms / wp_term_taxonomy / wp_term_relationships	Taxonomías estándar de WordPress cuando son necesarias.
Tablas de plugins	Action Scheduler, Hostinger Reach y LiteSpeed forman parte del entorno, no del modelo propio de NexaTech.

4.3 Páginas publicadas

ID	Título	Slug	Contenido
6	Servicios	servicios	Layout definido en page-servicios.php
10	Proceso	proceso	Layout definido en page-proceso.php
12	Casos de Estudio	casos-de-estudio	Layout definido en page-casos-de-estudio.php
14	Vantix Retail	vantix-retail	Contenido administrado desde WordPress
27	Stratum Solutions	stratum-solutions	Contenido administrado desde WordPress
37	Axionis Pharma	axionis-pharma	Contenido administrado desde WordPress
43	Contacto	contacto	Formulario definido en page-contacto.php

Inicio

La portada no necesita un registro propio con todo su contenido: front-page.php construye directamente el layout de Inicio.

4.4 Casos de estudio como páginas de WordPress

Los tres casos detallados se guardan como páginas publicadas. Su contenido principal vive en post_content y conserva bloques de WordPress con encabezados, párrafos, listas y separadores. page-articulo.php recupera ese contenido mediante el Loop y the_content().

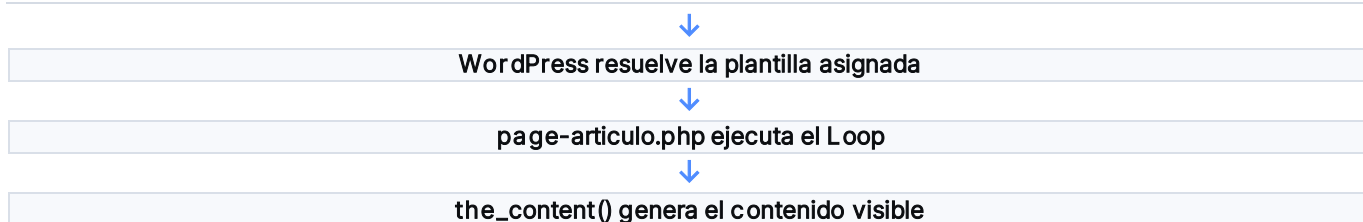
Administrador edita un caso



WordPress guarda el contenido en wp_posts



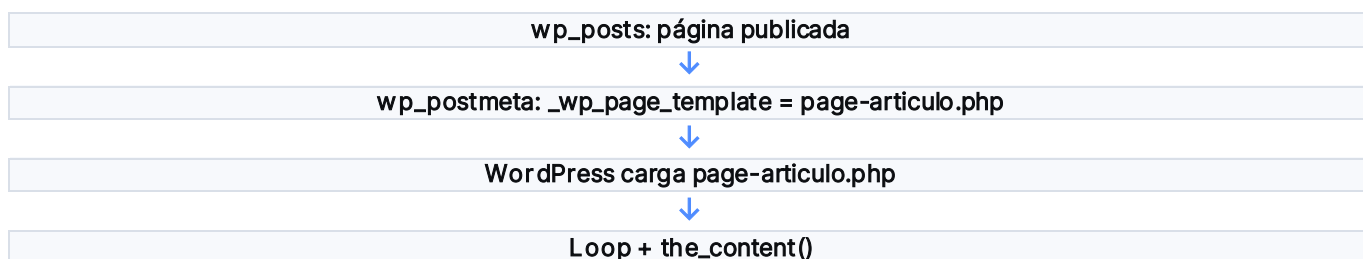
El visitante abre el caso



4.5 Cómo queda asignado page-articulo.php

WordPress guarda en `wp_postmeta` la clave `_wp_page_template` para recordar qué plantilla personalizada debe utilizar una página. Los tres casos tienen el valor `page-articulo.php`, por lo que la relación entre el contenido guardado y el template no depende del título ni de una condición escrita a mano dentro del archivo.

Caso	ID página	_wp_page_template
Vantix Retail	14	page-articulo.php
Stratum Solutions	27	page-articulo.php
Axionis Pharma	37	page-articulo.php



4.6 Imagen destacada y relación con attachments

Cada caso utiliza `_thumbnail_id` dentro de `wp_postmeta` para apuntar a un attachment de WordPress. El attachment, a su vez, representa el archivo WebP almacenado en `wp-content/uploads`.

Caso	ID página	_thumbnail_id	Attachment	Archivo
Vantix Retail	14	24	24	006_vantix_retail.webp
Stratum Solutions	27	22	22	003_stratum.webp
Axionis Pharma	37	23	23	004_axionis_pharma.webp

Relación de la imagen

Página del caso → `wp_postmeta: _thumbnail_id` → `wp_posts: attachment` → `_wp_attached_file` → archivo WebP en uploads.

Ejemplo completo · Vantix Retail

La página ID 14 guarda el contenido del caso. En `wp_postmeta`, `_wp_page_template = page-articulo.php` asigna la plantilla y `_thumbnail_id = 24` relaciona la página con el attachment 24. Ese attachment referencia `006_vantix_retail.webp`. Al abrir el caso, WordPress carga `page-articulo.php`, muestra la imagen destacada y procesa `post_content` mediante `the_content()`.

4.7 Metadatos de los attachments

WordPress conserva para los tres archivos originales un tamaño de 1024×1024 píxeles y referencias a variantes generadas automáticamente, como 150×150 , 300×300 y 768×768 . El archivo físico no se guarda dentro de la base: la base conserva la referencia y la metadata.

4.8 Opciones importantes de WordPress

Opción	Valor / función
--------	-----------------

Opción	Valor / función
siteurl / home	https://demo-nexatech.mcartworksstudio.com
template / stylesheet	nexatech-theme
current_theme	NexaTech Custom
permalink_structure	/%postname%/
active_plugins	5 plugins registrados en la instalación

Los cinco plugins registrados son Hostinger AI Assistant, Hostinger Easy Onboarding, Hostinger Reach, Hostinger y Media Sync. Forman parte de la instalación importada, pero el tema NexaTech no utiliza sus APIs para construir Inicio, Servicios, Proceso, Casos o Contacto; por eso no se consideran dependencias funcionales del tema.

4.9 No existe una tabla de consultas de contacto

El formulario actual no guarda sus datos en MariaDB. Los valores recibidos se sanitizan, se validan y se utilizan únicamente para decidir el estado de la interfaz. La integración con email, una tabla propia o un CRM queda como punto de ampliación.

5. Estructura del proyecto

El código específico se concentra en wp-content/themes/nexatech-theme. WordPress y los plugins pertenecen al entorno de ejecución; los templates y nexatech.css constituyen el núcleo desarrollado para NexaTech.

```

wp-content/
├── themes/
│   └── nexatech-theme/
│       ├── style.css
│       ├── functions.php
│       ├── header.php
│       ├── footer.php
│       ├── front-page.php
│       ├── page-servicios.php
│       ├── page-proceso.php
│       ├── page-casos-de-estudio.php
│       ├── page-articulo.php
│       ├── page-contacto.php
│       ├── index.php
│       └── nexatech.css
└── uploads/
    ├── 001_nordexia.webp
    ├── 002_helion_group.webp
    ├── 003_stratum.webp
    ├── 004_axionis_pharma.webp
    ├── 005_corevia_motors.webp
    └── 006_vantix_retail.webp
  
```

Archivo	Responsabilidad
style.css	Registra ante WordPress el tema "NexaTech Custom", su versión y datos básicos.
functions.php	Configuración global, CSS, featured images y optimizaciones de WordPress.
header.php	Documento HTML, Google Fonts, AOS CSS y navegación global.
footer.php	Pie del sitio, crédito y wp_footer().
front-page.php	Inicio, Typed.js, métricas e Isotope.
page-servicios.php	Servicios, métricas y bloques alternados.
page-proceso.php	Timeline y métricas simples/de rango.
page-casos-de-estudio.php	Métricas, logos con Splide, tarjetas y CTA.
page-articulo.php	Loop de WordPress, imagen destacada y contenido de los casos.
page-contacto.php	Procesamiento POST, validación, mensajes y formulario.
index.php	Fallback mínimo de la jerarquía de templates; no construye una pantalla propia.
nexatech.css	Hoja principal de estilos; 1.546 líneas en la versión suministrada.

6. WordPress y el tema personalizado

NexaTech utiliza un tema propio, no un tema hijo. Tanto template como stylesheet están configurados como nexatech-theme. Esto significa que la estructura principal no depende de Blocksy ni de otro tema padre para construir las páginas del proyecto.

6.1 Registro del tema mediante style.css

```

/*
Theme Name: NexaTech Custom
Author: Gaston Rizzo
Version: 1.0
*/
  
```

WordPress reconoce un tema por el encabezado de style.css. En NexaTech ese archivo registra el nombre visible "NexaTech Custom", el autor y la versión 1.0. La hoja visual principal del sitio no es style.css sino nexatech.css, que se encola desde functions.php.

6.2 Jerarquía de templates e index.php

WordPress aplica su jerarquía de templates para decidir qué archivo debe construir cada URL. NexaTech define archivos específicos para las páginas principales. index.php permanece como fallback mínimo y contiene únicamente el comentario “Silence is golden.”; por eso no genera una pantalla propia mientras exista un template más específico.

6.3 Carga global mediante header y footer

```
<?php get_header(); ?>

<!-- contenido del template -->

<?php get_footer(); ?>
```

header.php abre el documento HTML, carga la fuente Inter, incorpora el CSS de AOS y ejecuta wp_head(). footer.php cierra la interfaz, muestra el crédito y ejecuta wp_footer() antes de cerrar body y html.

6.4 Navegación activa

```
<a href="<?php echo home_url('/servicios/'); ?>"
  class="<?php echo is_page('servicios') ? 'active' : ''; ?>">
  Servicios
</a>
```

is_front_page() e is_page() permiten que el menú marque visualmente la sección actual sin mantener estados manuales en JavaScript.

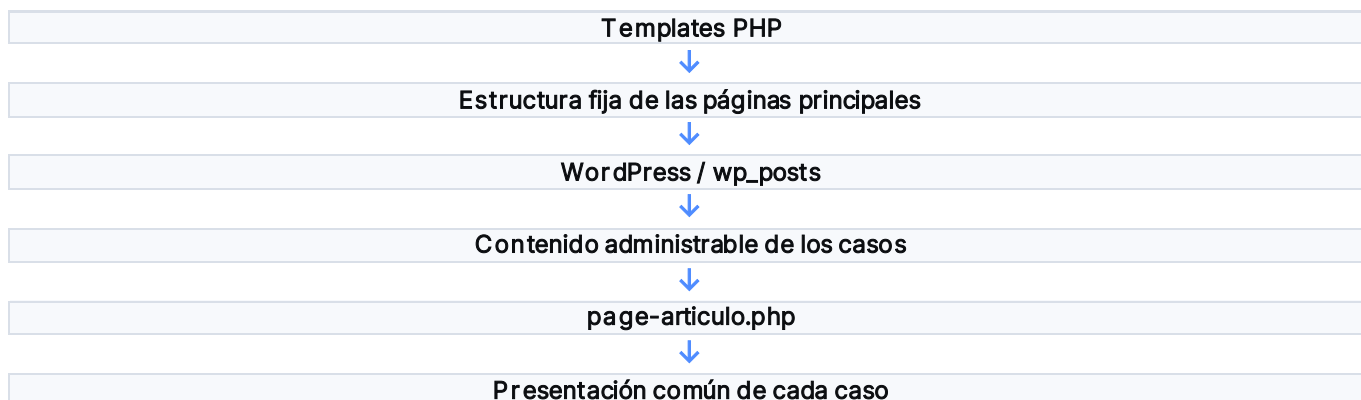
6.5 Imágenes destacadas

```
add_theme_support(
  'post-thumbnails',
  array('page', 'post')
);
```

Esta configuración habilita featured images para páginas y entradas. Es necesaria porque los casos Vantix, Stratum y Axionis muestran una imagen destacada antes de su contenido.

6.6 Qué administra WordPress y qué queda en los templates

Inicio, Servicios, Proceso, la página general de Casos y el formulario de Contacto tienen gran parte de su estructura y texto definidos directamente en sus templates PHP. Los tres casos individuales, en cambio, guardan su contenido en WordPress y utilizan page-articulo.php como plantilla común.



6.7 Filtros de Blocksy presentes en page-articulo.php

```
add_filter('blocksy:page:title:enabled', '__return_false');
add_filter('blocksy:single:has-hero', '__return_false');
```

page-articulo.php conserva dos filtros cuyos nombres pertenecen a Blocksy. Sin embargo, la instalación activa nexatech-theme como template y stylesheet, y el resto del proyecto no utiliza Blocksy para construir sus páginas. Estos filtros quedaron como compatibilidad residual de una implementación anterior: registrar el callback no convierte a Blocksy en una dependencia funcional del tema actual.

7. Página de Inicio

front-page.php construye la portada completa. Reúne un hero, métricas, servicios resumidos, las cuatro etapas del proceso y una grilla de casos filtrable. La página carga AOS, Isotope y Typed.js desde CDN y utiliza JavaScript propio para sincronizar las métricas con las animaciones de entrada.

7.1 Hero principal y texto dinámico

```
<h1>
  Construimos<br>
  <span id="typed"></span><br>
  que escalan.
</h1>
```

Typed.js escribe sucesivamente “empresas B2B”, “plataformas web” y “productos digitales”. El texto se borra y vuelve a escribirse en bucle.

```
new Typed('#typed', {
  strings: [
    'empresas B2B',
    'plataformas web',
    'productos digitales'
  ],
  typeSpeed: 50,
  backSpeed: 25,
  backDelay: 1200,
  loop: true,
  smartBackspace: true
});
```

7.2 Métricas del hero

Métrica	Valor objetivo
Performance promedio	98/100
Proyectos entregados	120+
Clientes B2B	40+

Los números comienzan en cero y se incrementan con requestAnimationFrame(). El script espera a que finalice la entrada AOS del bloque visual antes de iniciar los contadores.

7.3 Servicios y proceso resumidos

Inicio presenta cuatro tarjetas de servicios —Desarrollo Web, Arquitectura Frontend, Automatización y Consultoría Técnica— y cuatro pasos de proceso —Análisis, Estrategia, Desarrollo y Optimización—. Cada bloque incorpora data-aos y delays progresivos.

7.4 Casos filtrables con Isotope

La grilla de Inicio contiene casos de ejemplo clasificados mediante clases como web, frontend y automation. Los botones de filtro entregan a Isotope un selector y la librería oculta o reorganiza los elementos sin recargar la página.

```
var iso = new Isotope(grid, {
  itemSelector: '.case-item',
  layoutMode: 'fitRows'
});

iso.arrange({ filter: filterValue });
```

Usuario pulsa Web / Frontend / Automatización



JavaScript lee data-filter



Isotope aplica el filtro



La grilla se reorganiza sin recarga

8. Página de Servicios

page-servicios.php desarrolla las áreas de trabajo de la consultora mediante un hero con métricas y cuatro bloques alternados. Los bloques cambian la dirección de entrada para producir un ritmo visual izquierda/derecha durante el scroll.

8.1 Métricas del encabezado

Métrica	Valor
Años de experiencia técnica	10+
Sistemas en producción	40+
Entregables medibles	100%

8.2 Bloques de servicio

N.º	Servicio	Aspectos destacados
01	Desarrollo Web	Arquitectura escalable, performance, accesibilidad, SEO técnico e integraciones.
02	Arquitectura Frontend	Separación de responsabilidades, escalabilidad y decisiones técnicas conscientes.
03	Automatización	Flujos internos, reducción de tareas manuales y optimización de procesos.
04	Integraciones & Optimización	Conexión entre plataformas, auditorías técnicas y mejora continua.

8.3 Alternancia visual y AOS

```
<div class="service-block reverse">
  <div class="service-content"
    data-aos="fade-left"
    data-aos-delay="120">
    ...
  </div>
</div>
```

Los bloques normales entran desde la derecha de la pantalla mediante fade-right y los bloques reverse utilizan fade-left. Los delays 0, 120, 240 y 360 ms escalonan la secuencia.

9. Página de Proceso

page-proceso.php representa la metodología de trabajo como un timeline vertical. El encabezado incorpora métricas y cada etapa alterna su entrada visual mediante AOS.

9.1 Las cuatro fases

Fase	Descripción
01 · Análisis	Comprender negocio, contexto y restricciones técnicas.
02 · Estrategia	Definir arquitectura priorizando impacto y sostenibilidad.
03 · Desarrollo	Construir con foco en performance, calidad y buenas prácticas.
04 · Optimización	Medir, iterar y mejorar continuamente.

9.2 Métricas simples y métricas de rango

El template combina contadores simples con una métrica de rango. Las iteraciones se definen con `data-min="6"` y `data-max="12"`, por lo que JavaScript anima dos números en paralelo hasta mostrar 6–12.

```
<div class="metric-value metric-range"
  data-min="6"
  data-max="12">
  <span class="min">0</span>–<span class="max">0</span>
</div>
```

El resto de las métricas utiliza `data-target="4"` para las fases y `data-target="100"` para los entregables medibles.

10. Casos de estudio y artículos

NexaTech utiliza dos niveles para presentar casos. `page-casos-de-estudio.php` construye la página general con métricas, logos y tarjetas. Los casos detallados se administran como páginas de WordPress y se muestran mediante `page-articulo.php`.

10.1 Página general de casos

Bloque	Contenido
Hero	Métricas sobre mantenibilidad, tiempos de carga y resultados.
Confianza / logos	Carrusel continuo con seis marcas.
Casos destacados	Vantix Retail, Stratum Solutions y Axionis Pharma.
CTA	Enlace hacia Contacto.

10.2 Carrusel de logos con Splide

Los seis logos se referencian mediante `content_url()` desde `wp-content/uploads` y se colocan dentro de una lista de slides. Splide se configura como loop, con ancho automático, sin flechas ni paginación y con Auto Scroll continuo.

```
new Splide('.cases-logos', {
  type: 'loop',
  drag: false,
  arrows: false,
  pagination: false,
  autoWidth: true,
  gap: '72px',
  autoScroll: {
    speed: 0.7,
    pauseOnHover: true,
    pauseOnFocus: false
  }
}).mount(window.splide.Extensions);
```

10.3 Empresas mostradas en el carrusel

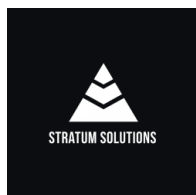
Orden	Marca / archivo
1	Nordexia · 001_nordexia.webp
2	Helion Group · 002_helion_group.webp
3	Stratum Solutions · 003_stratum.webp
4	Axionis Pharma · 004_axionis_pharma.webp

Orden	Marca / archivo
5	Corevia Motors · 005_corevia_motors.webp
6	Vantix Retail · 006_vantix_retail.webp

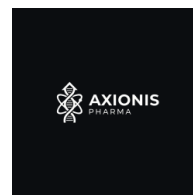
10.4 Tres casos desarrollados



Vantix Retail



Stratum Solutions



Axionis Pharma

Caso	Tema central	Resultado presentado
Vantix Retail	Optimización de plataforma e-commerce	–42% en tiempos de carga y mejora de la experiencia de compra.
Stratum Solutions	Escalabilidad frontend para plataforma financiera	Mayor velocidad de desarrollo y una base preparada para crecer.
Axionis Pharma	Automatización de procesos internos críticos	Ahorro operativo, trazabilidad y mayor control regulatorio.

10.5 Rutas de los casos y slugs de WordPress

Las tarjetas de `page-casos-de-estudio.php` escriben directamente las rutas de destino. La base de WordPress registra, por separado, los slugs de las tres páginas. Para que una tarjeta resuelva el caso correcto sin reglas adicionales, ambos valores deben coincidir con la estructura de enlaces permanentes del sitio.

Caso	Ruta escrita en la tarjeta	Slug registrado
Vantix Retail	<code>/casos/vantix-retail</code>	<code>vantix-retail</code>
Stratum Solutions	<code>/casos/stratum-solution</code>	<code>stratum-solutions</code>
Axionis Pharma	<code>/casos/axionis-pharma</code>	<code>axionis-pharma</code>

Punto a tener en cuenta

Las rutas de las tarjetas incluyen el segmento `/casos/`, mientras las páginas están registradas como páginas raíz. Además, Stratum usa “stratum-solution” en el href y “stratum-solutions” como slug. Si no existe una regla externa que traduzca esas direcciones, conviene alinear los enlaces del template con los permalinks reales de WordPress.

10.6 `page-articulo.php` y el Loop de WordPress

```
<?php if ( have_posts() ) : while ( have_posts() ) : the_post(); ?>

    <?php if ( has_post_thumbnail() ) : ?>
        <?php the_post_thumbnail('large'); ?>
    <?php endif; ?>

    <?php the_content(); ?>

<?php endwhile; endif; ?>
```

El template no necesita conocer de antemano el texto de Vantix, Stratum o Axionis. WordPress entrega la página solicitada; `the_post_thumbnail()` muestra la imagen destacada y `the_content()` procesa el contenido almacenado en la base.

Recorrido del caso

Editor actualiza el caso en WordPress → `wp_posts` conserva el contenido → `wp_postmeta` relaciona la imagen → `page-articulo.php` ejecuta el Loop → el visitante recibe el caso completo.

10.7 AOS en los artículos

La imagen destacada entra con fade-up y el contenido utiliza el mismo efecto con un delay de 150 ms. En esta plantilla AOS se configura con duration 700, once true y offset 200, por lo que la animación no se repite al volver a recorrer la página.

10.8 Ejemplo completo: Vantix Retail



Este recorrido muestra la separación entre contenido y presentación. Vantix conserva su texto en la base, la plantilla común define cómo se presenta y el attachment aporta la imagen destacada. Stratum Solutions y Axionis Pharma utilizan la misma estructura con sus propios IDs y archivos.

11. Formulario de contacto

page-contacto.php combina la interfaz del formulario con su procesamiento en el servidor. El bloque PHP se ejecuta antes de cargar el encabezado y sólo procesa datos cuando REQUEST_METHOD es POST.

11.1 Campos

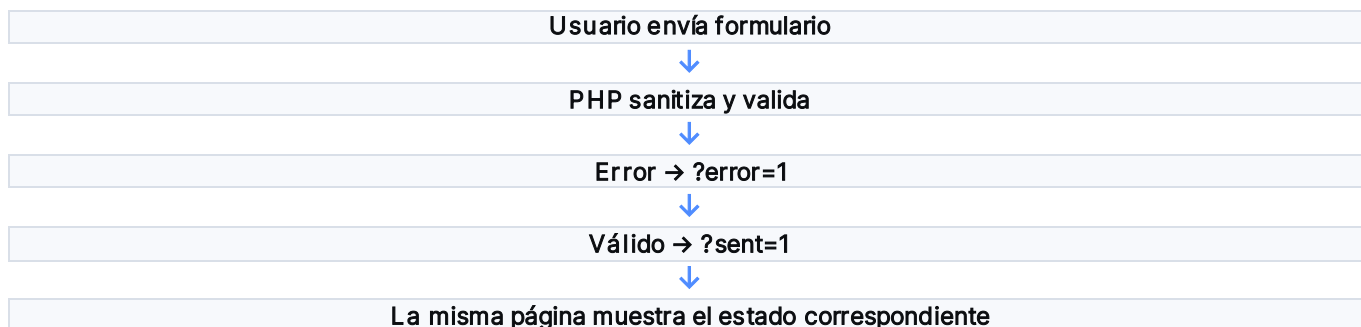
Campo	HTML	Tratamiento PHP
Nombre	text + required	sanitize_text_field()
Apellido	text + required	sanitize_text_field()
Email corporativo	email + required	sanitize_email() + is_email()
Empresa	text + required	sanitize_text_field()
Mensaje	textarea + required	sanitize_textarea_field()

11.2 Validación en dos niveles

required y type="email" ayudan al usuario desde el navegador, pero no sustituyen la validación del servidor. PHP vuelve a comprobar que todos los campos estén presentes y que el email tenga un formato válido.

```
if (
    empty($first_name) ||
    empty($last_name) ||
    empty($email) ||
    empty($company) ||
    empty($message) ||
    ! is_email($email)
) {
    wp_redirect(
        add_query_arg('error', '1', get_permalink())
    );
    exit;
}
```

11.3 Redirección después del POST



La redirección separa la petición POST original de la pantalla final. Al recargar después de una respuesta exitosa, el navegador ya está en una dirección GET con sent=1 y no vuelve a enviar automáticamente los datos del formulario.

11.4 Estado visual

Con sent=1 se oculta el formulario, se muestra el mensaje “Gracias, recibimos tu consulta. Te responderemos a la brevedad.” y aparece un botón para volver al inicio. Con error=1 se mantiene el formulario y se informa que los campos deben completarse correctamente.

11.5 Alcance actual

Importante

La versión actual valida y presenta estados de éxito/error, pero no envía un correo ni guarda la consulta. El código deja expresamente preparado el punto para integrar wp_mail(), almacenamiento en base de datos o un CRM.

12. JavaScript e interacciones dinámicas

NexaTech utiliza JavaScript vanilla para los comportamientos propios y delega tareas específicas a cuatro librerías externas. El código se incluye dentro de los templates donde cada interacción es necesaria.

12.1 AOS.js

AOS interpreta atributos data-aos y data-aos-delay colocados en el HTML. Inicio, Servicios, Proceso, Casos, Contacto y los artículos inicializan la librería con configuraciones similares, aunque la duración, el offset y la repetición cambian según la página.

Página	Duración	Repetición / comportamiento
Inicio	800 ms	once false; offset 80
Servicios	900 ms	animaciones repetibles durante el scroll
Proceso	900 ms	once true; offset 200
Casos	900 ms	animaciones repetibles
Contacto	900 ms	animaciones repetibles
Artículo	700 ms	once true; offset 200

12.2 Typed.js

Se utiliza únicamente en el hero de Inicio. El efecto se limita a un span con id typed, por lo que el resto del h1 permanece estable mientras cambia el término central.

12.3 Isotope.js

Isotope trabaja sobre .cases-grid y considera .case-item como unidad. Los botones de filtro actualizan la clase active y llaman a arrange() con el valor almacenado en data-filter.

12.4 Splide.js y Auto Scroll

Splide aparece en la página Casos de estudio. El CSS se encola desde functions.php sólo cuando is_page("casos-de-estudio") es verdadero, mientras que el JavaScript de Splide y la extensión Auto Scroll se cargan desde el propio template.

12.5 requestAnimationFrame() para métricas

Los contadores de Inicio, Servicios, Proceso y Casos no cambian de 0 al valor final de manera instantánea. El código incrementa el valor en cada frame y solicita al navegador el siguiente ciclo mediante requestAnimationFrame().

```
function animate() {
    current += speed;

    if (current < target) {
        numberEl.textContent = Math.ceil(current);
        requestAnimationFrame(animate);
    } else {
        numberEl.textContent = target;
    }
}
```

Este mecanismo sincroniza la actualización con el ciclo de repintado del navegador y evita depender de un setInterval fijo para cada número.

13. CSS y adaptación de la interfaz

nexatech.css concentra la presentación del proyecto. La versión suministrada tiene 1.546 líneas y define variables globales, header, footer, hero, servicios, timeline, casos, formularios y reglas responsive.

13.1 Variables globales

```
:root {
  --bg-dark: #0b0d10;
  --bg-light: #12151b;
  --text-main: #ffffff;
  --text-muted: #9aa3b2;
  --accent: #4f8cff;
}
```

El azul #4f8cff actúa como acento principal. Los fondos oscuros y los textos claros se reutilizan mediante variables para mantener consistencia entre las páginas.

13.2 Contenedor principal

Las páginas principales utilizan un wrapper con ancho máximo de 1400 px, gradiente radial, bordes redondeados y márgenes calculados con clamp(). Los contenedores internos usan aproximadamente 90% del ancho, con un máximo de 1300 px.

13.3 Header sticky

El encabezado permanece visible con position: sticky, fondo semitransparente, backdrop-filter blur y una línea inferior tenue. La navegación usa color muted para enlaces inactivos y texto blanco para el enlace activo.

13.4 Responsive

La hoja incluye media queries en 1600, 1100, 900, 768 y 600 px. La mayor parte de los cambios importantes ocurre en 900 px, donde grids, navegación, footer y bloques alternados pasan a distribuciones más simples.

Breakpoint	Uso principal
1600 px	Ajustes finos del footer en pantallas grandes/medias.
1100 px	Adaptaciones de bloques de casos y layouts intermedios.
900 px	Cambio principal hacia tablet/móvil: grids, navegación, servicios y footer.
768 px	Ajustes adicionales de contenido y formularios.
600 px	Configuraciones específicas para móviles angostos.

14. Recursos multimedia y servicios externos

14.1 Archivos propios

Los logos corporativos y las imágenes destacadas se encuentran dentro de wp-content/uploads. Los tres casos administrados por WordPress tienen además attachments registrados en la base y variantes generadas por el CMS.

Recurso	Origen
Logos de empresas	wp-content/uploads/*.webp
Imágenes destacadas de casos	Attachments de WordPress + archivos WebP
CSS principal	Archivo local nexatech.css
Templates PHP	Tema local nexatech-theme

14.2 Servicios externos

Servicio	Qué entrega
Google Fonts	Tipografía Inter.
unpkg	AOS 2.3.4 y AOS.css; Isotope Layout 3 en Inicio.
jsDelivr	Typed.js 2.0.12 y Splide 4.1.4.

Servicio	Qué entrega
Splide Auto Scroll	Extensión 0.5.3 para el desplazamiento continuo del carrusel.

Si estos servicios externos no están disponibles, el HTML y el CSS local del tema siguen existiendo, pero se pierden las funciones asociadas a la librería que no pudo descargarse: animaciones AOS, efecto Typed, filtros Isotope o carrusel Splide.

15. Caché y rendimiento

15.1 Versionado dinámico de nexatech.css

```
wp_enqueue_style(
    'nexatech-main',
    $css_uri,
    [],
    filemtime($css_path),
    'all'
);
```

filemtime() utiliza la fecha de modificación del archivo como versión. Al cambiar nexatech.css cambia también la URL generada por WordPress y el navegador puede solicitar la versión nueva.

15.2 Carga condicional de Splide CSS

functions.php sólo encola la hoja de estilos oficial de Splide cuando is_page("casos-de-estudio") es verdadero. Las demás páginas no necesitan descargar ese CSS.

15.3 Recursos de WordPress eliminados del frontend

- Scripts y estilos de emojis.
- WordPress Block Library y estilos clásicos de bloques.
- Global Styles de WordPress.
- Dashicons para visitantes no autenticados.
- Admin Bar del frontend.
- Atributo automático sizes para imágenes mediante wp_img_tag_add_auto_sizes.

15.4 Bloques Gutenberg sin los estilos estándar de WordPress

Los casos almacenados en post_content contienen bloques de Gutenberg, por ejemplo wp-block-heading, wp-block-list y wp-block-separator. functions.php retira wp-block-library, classic-theme-styles y Global Styles del frontend. El HTML de esos bloques sigue siendo generado por the_content(), pero su presentación depende del CSS propio de NexaTech en lugar de las hojas estándar de WordPress.

15.5 No-cache durante desarrollo

```
header('Cache-Control: no-cache, no-store, must-revalidate');
header('Pragma: no-cache');
header('Expires: 0');
```

La función force_nocache_for_dev() se ejecuta para páginas del frontend y obliga a servir una versión reciente del HTML durante el desarrollo. Esto resulta útil mientras se modifican templates, aunque la política de caché puede separarse por entorno si el proyecto se despliega con una estrategia de producción diferente.

16. Tratamiento de datos y seguridad

El área donde NexaTech recibe datos directos del visitante es el formulario de contacto. La implementación combina restricciones del navegador con sanitización y validación en el servidor.

Medida	Uso
required	Evita envíos incompletos desde la interfaz normal.
type="email"	Solicita al navegador validar formato básico de email.

Medida	Uso
<code>sanitize_text_field()</code>	Limpia nombre, apellido y empresa.
<code>sanitize_email()</code>	Normaliza el email recibido.
<code>sanitize_textarea_field()</code>	Limpia el mensaje conservando el carácter de texto multilínea.
<code>is_email()</code>	Valida nuevamente el formato del email en PHP.
<code>wp_redirect()</code>	Redirige mediante funciones de WordPress en vez de construir headers manuales.
<code>esc_url()</code>	Se utiliza al imprimir la URL del botón de regreso al Inicio.

16.1 Por qué el servidor valida de nuevo

Las reglas HTML pueden modificarse desde las herramientas del navegador o una petición puede enviarse sin utilizar el formulario visible. Por eso PHP no confía en `required` y repite las comprobaciones antes de continuar.

16.2 Nonce del formulario

El formulario actual no incorpora un nonce propio de WordPress. La validación y la sanitización existentes controlan el formato de los datos, pero un nonce agrega una comprobación adicional de origen/intención para acciones POST. Si el formulario se conecta en el futuro con `wp_mail()`, un CRM o almacenamiento persistente, conviene incorporar `wp_nonce_field()` y verificarlo en el procesamiento.

16.3 Configuración sensible

Las credenciales de base de datos, claves y contraseñas pertenecen a la configuración del entorno y no forman parte del código público del tema. Para trasladar el proyecto a otro servidor deben definirse en `wp-config.php` del nuevo entorno.

17. Instalación y ejecución del proyecto

Para reproducir NexaTech se necesitan la instalación de WordPress, la base de datos y los archivos del tema y de uploads. El tema no depende de Blocksy ni de WooCommerce.

Paso	Acción
1	Instalar WordPress en el servidor o entorno local.
2	Crear una base MariaDB/MySQL e importar <code>u679645666_QLcZe.sql</code> .
3	Configurar <code>wp-config.php</code> con nombre de base, usuario, contraseña y host del nuevo entorno.
4	Copiar <code>nexatech-theme</code> dentro de <code>wp-content/themes/</code> .
5	Copiar los archivos multimedia del proyecto dentro de <code>wp-content/uploads/</code> .
6	Activar "NexaTech Custom" desde Apariencia → Temas.
7	Actualizar <code>siteurl</code> y <code>home</code> si cambia el dominio o la ruta de la instalación.
8	Revisar <code>/%postname%/</code> y regenerar enlaces permanentes si es necesario.
9	Comprobar Inicio, Servicios, Proceso, Casos de estudio, Contacto y los tres casos individuales.
10	Verificar que el entorno tenga salida a Internet para Google Fonts, AOS, Typed, Isotope y Splide.

17.1 Plugins registrados y dependencias reales

La base importada registra cinco plugins: Hostinger AI Assistant, Hostinger Easy Onboarding, Hostinger Reach, Hostinger y Media Sync. El código de `nexatech-theme` no llama a sus APIs para construir las páginas principales. Por eso pueden formar parte del entorno original sin ser dependencias funcionales del tema. Las dependencias frontend reales son las librerías cargadas por el propio código: AOS, Typed.js, Isotope y Splide.

17.2 Qué recupera la base

- Las siete páginas publicadas del sitio.
- El contenido completo de Vantix Retail, Stratum Solutions y Axionis Pharma.
- Los tres attachments utilizados como imágenes destacadas y sus metadatos.
- Las opciones de WordPress, el tema activo y los plugins registrados.
- La cuenta administrativa y la configuración general del CMS.

17.3 Qué recuperan los archivos

- Los templates PHP que construyen las páginas.
- nexatech.css con todo el sistema visual.
- Los seis logos WebP y las variantes de las imágenes registradas en WordPress.

Base de datos y uploads cumplen funciones distintas

MariaDB conserva registros, relaciones y configuración. wp-content/uploads contiene los archivos de imagen que esos registros referencian.

18. Posibles líneas de evolución

La arquitectura actual cubre la presentación de la empresa, los casos y el contacto definidos para NexaTech. A partir de esta base, el proyecto puede ampliarse sin cambiar su enfoque general.

Área	Posible evolución
Contacto	Conectar el punto preparado con wp_mail(), un CRM o almacenamiento propio.
Contenido	Mover más textos estructurales desde los templates hacia campos administrables en WordPress.
Casos de estudio	Crear un tipo de contenido específico si el volumen de casos creciera y necesitara taxonomías propias.
Assets externos	Agrupar o alojar localmente librerías si se necesitara mayor control sobre versiones y disponibilidad.
Entornos	Separar explícitamente políticas de caché para desarrollo y producción.
Accesibilidad y SEO	Ampliar atributos semánticos, metadatos y revisiones de accesibilidad a medida que crezca el sitio.

19. Alcance técnico y conclusión

NexaTech combina WordPress con un tema personalizado para construir un sitio corporativo donde el código mantiene un control amplio sobre la presentación, mientras el CMS conserva páginas, casos, imágenes destacadas y configuración general.

Qué demuestra el proyecto

- Creación y activación de un tema WordPress propio.
- Comprensión de la jerarquía de templates, el registro del tema mediante style.css y el uso de get_header(), get_footer(), wp_head() y wp_footer().
- Uso de páginas y metadatos estándar de WordPress sin crear tablas propias.
- Renderizado de contenido administrado mediante _wp_page_template, el Loop, the_content() y featured images.
- Construcción de layouts propios con PHP, HTML y una hoja CSS central.
- Integración de AOS, Typed.js, Isotope y Splide con JavaScript vanilla.
- Animación eficiente de métricas mediante requestAnimationFrame().
- Formulario con validación frontend, sanitización backend y patrón POST → redirect → GET.
- Carga condicional y limpieza de recursos de WordPress para reducir elementos innecesarios.
- Diseño adaptable mediante variables CSS, grids y media queries.

Cierre

El resultado es una aplicación web corporativa en la que WordPress funciona como infraestructura y CMS, mientras nexatech-theme define la identidad visual del sitio. La base de datos conserva los contenidos administrables; PHP resuelve las plantillas; CSS construye la interfaz; y JavaScript agrega las interacciones dinámicas que completan la experiencia del sitio.